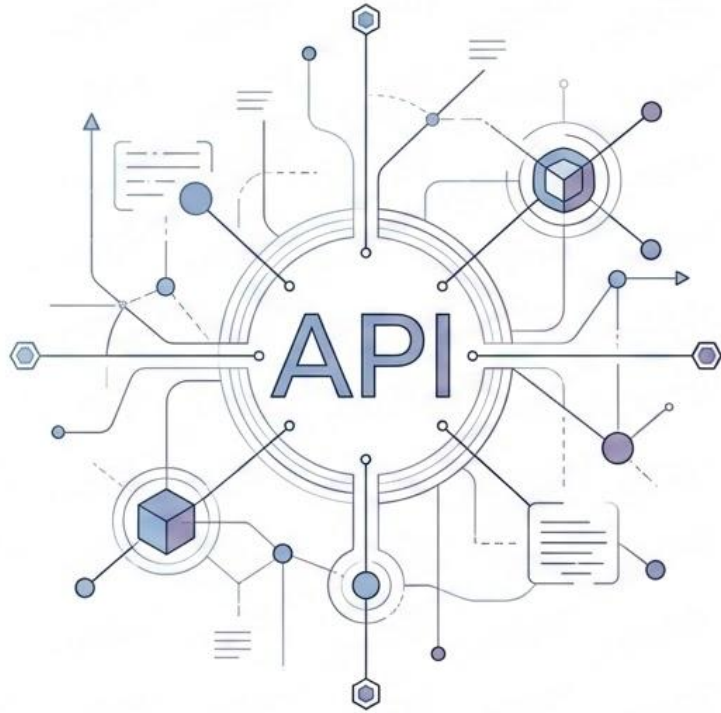




Module 7: Introduction to RESTful API Development with ASP.NET Core

Building clean, scalable, and testable web APIs

Learning Objectives

By the end of this module, students will be able to:



Understand what RESTful APIs are.



Explain how HTTP methods are used.



Identify the basic structure of an ASP.NET Core Web API project.



Create controllers, routes, and endpoints.



Work with requests, responses, and status codes.

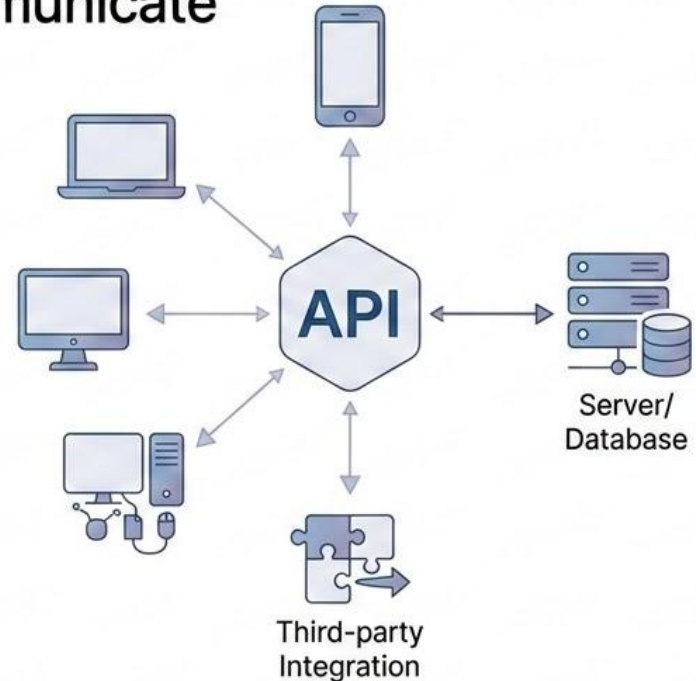


Understand the importance of validation, documentation, and testing.

Why APIs Matter

APIs allow different applications to communicate with each other, acting as a crucial intermediary for data and functionality.

- Web applications
- Mobile applications
- Desktop systems
- Third-party integrations
- Microservices



What Is a RESTful API?

A RESTful API is a web service that exposes resources through URLs and uses HTTP methods to perform operations.



Resources



HTTP



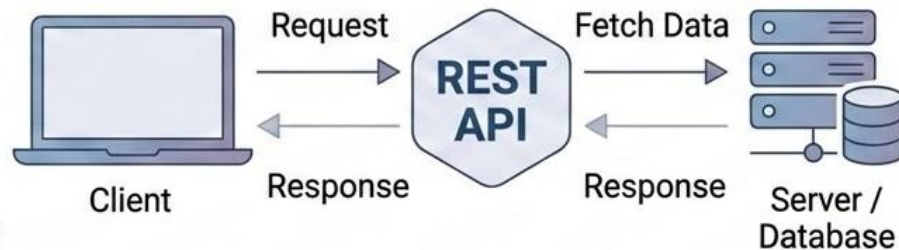
JSON



Stateless communication



Standard operations

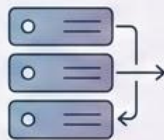


REST Core Principles



1. Client-Server Separation

The user interface (client) and data storage (server) (server) operate independently.



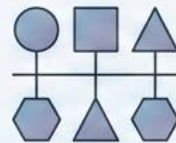
2. Stateless Communication

Each client request contains all necessary information; no server session state is stored.



3. Resource-Based Design

Key pieces of information are exposed as individual, identifiable entities.



4. Uniform Interface

Standard protocols and data formats create predictable access to all resources.



5. Cacheable Responses

Data can be stored by the client or intermediates to improve speed.

Resources and Endpoints

REST APIs are organized around logical resources, which represent specific data entities within the system.

`/api/products`
`/api/products/5`
`/api/users`
`/api/orders`

`/api/products`
`/api/products/5`
`/api/users`
`/api/orders`

Good

`/api/products`

Standard, resource-based URIs.

Avoid

`/api/getProducts`

Action-oriented URIs that embed verbs.

REST endpoints should identify resources, not actions, and usually employ nouns rather than verbs.

HTTP Methods



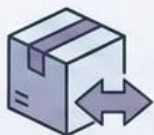
GET

Reads data from the specified resource.
Safe and Idempotent.



POST

Replaces the existing data with the provided payload. Idempotent.



PUT

Replaces the existing data with parent resource. Not Idempotent.



PATCH

Applies partial updates to a resource. Often not Idempotent.



PATCH

Applies's partial updates.
otennot Idempotent.



DELETE

Removes the specified resource.
Idempotent.

Example: Products API



GET /api/products — Get all products



GET /api/products/5 — Get product by ID



POST /api/products — Create product



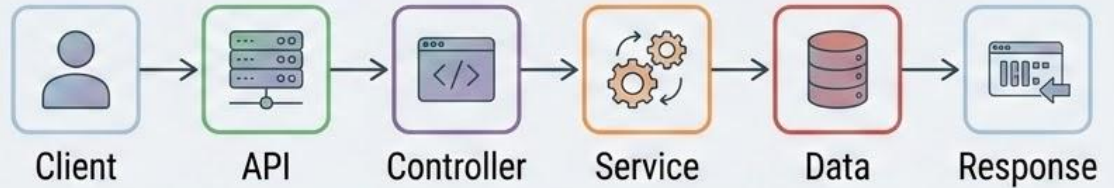
PUT /api/products/5 — Update product



DELETE /api/products/5 — Delete product

API Request and Response Flow

- Client sends an HTTP request.
- ASP.NET Core API receives the request.
- Controller processes the request.
- Service handles business logic.
- Data is returned.
- API sends an HTTP response.



HTTP Status Codes

Success (200 OK, 201 Created, 204 No Content)

- 200 OK
- 201 Created
- 204 No Content

Client Errors (400 Bad Request, 401 Unauthorized, 404 Not Found)

- 400 Bad Request
- 401 Unauthorized
- 404 Not Found

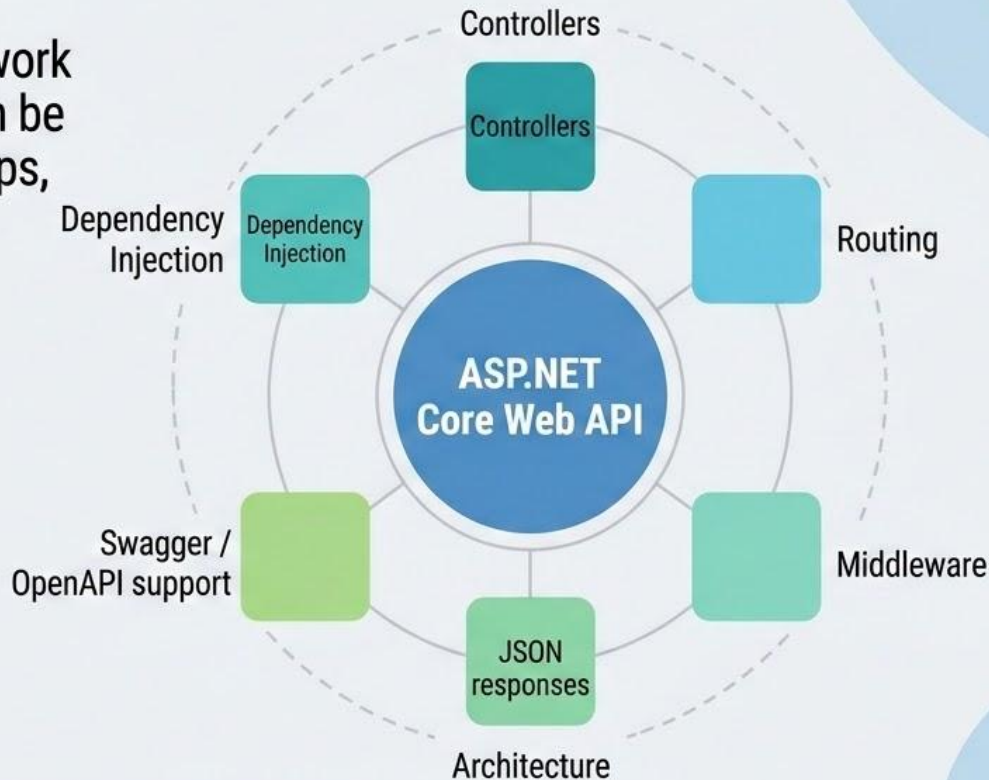
Server Error (500 Internal Server Error)

- 500 Internal Server Error

ASP.NET Core Web API

ASP.NET Core Web API is a framework for building HTTP services that can be consumed by web apps, mobile apps, and other systems.

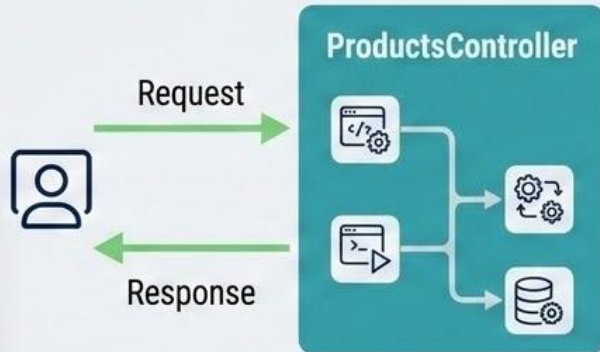
- Controllers
- Routing
- Dependency Injection
- Middleware
- JSON responses
- Swagger / OpenAPI support



Controllers and Actions

Controllers receive HTTP requests, and actions execute specific operations.
Example: ProductsController handles product-related endpoints.

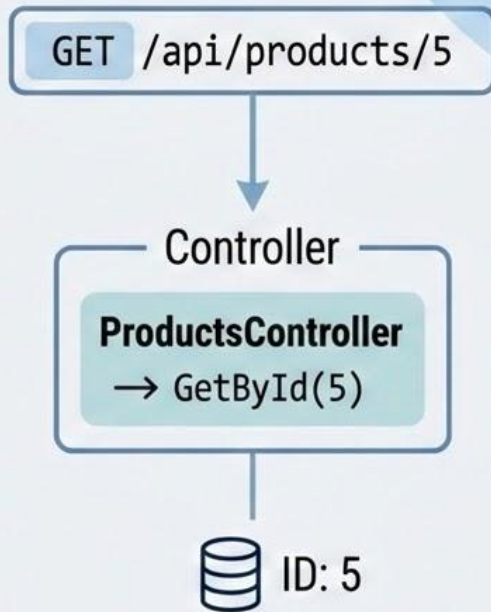
```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    // Actions defined here...
}
```



Routing in ASP.NET Core

- Routing connects URLs to controller actions.

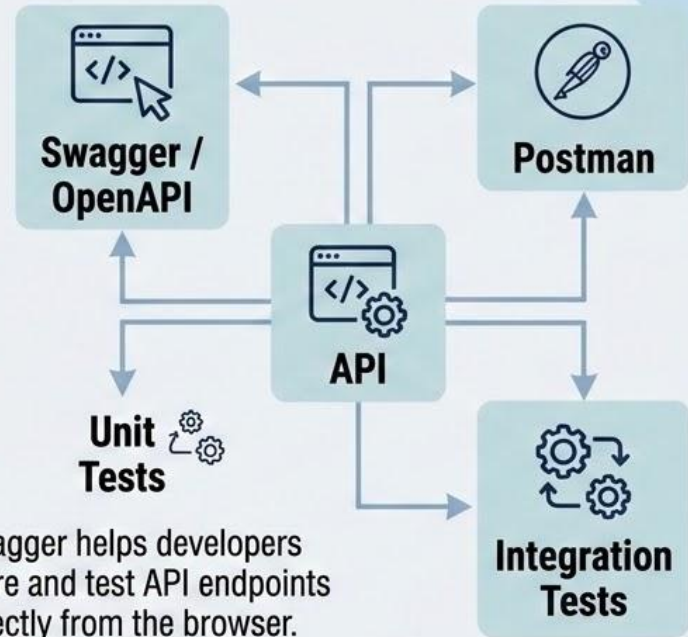
```
[ApiController]
[Route("api/[controller]")]
[HttpGet]
[HttpGet("{id}")]
[HttpPost]
```



Documentation and Testing

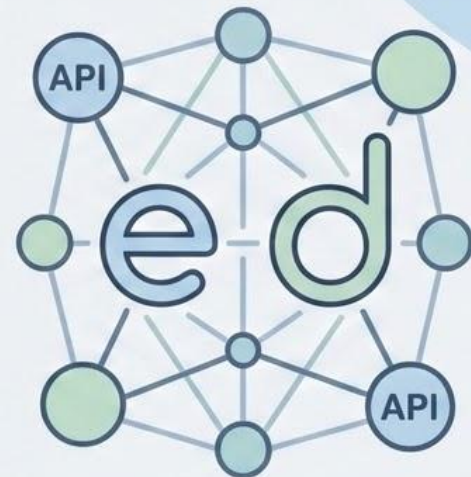
Common tools and practices used for documenting and testing ASP.NET Core APIs.

- Swagger / OpenAPI for documentation and interactive testing.
- Postman for manual API request testing.
- Unit Tests for isolated logic validation.
- Integration Tests for verifying full request-response flow.



Key Takeaways

-  REST APIs expose resources through URLs.
-  HTTP methods define the operation.
-  Status codes communicate the result.
-  ASP.NET Core Web API uses controllers, routing, services, and JSON.
-  Good APIs are validated, documented, tested, and easy to consume.



“Build APIs that are simple to consume and easy to maintain.”